WNDRVR
SUPPORT NETWORK

# VXWORKS 7 CRYPTOGRAPHY LIBRARIES PROGRAMMER'S GUIDE, SR0650

**Copyright Notice**

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

**Corporate Headquarters**

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

*VxWorks 7 Cryptography Libraries Programmer's Guide, SR0650*

16 September 2020

# 1. OVERVIEW

The VxWorks® 7 cryptography libraries provide standard cryptographic algorithms and supporting utilities.

They are based on the cryptographic portions of the open source project OpenSSL. They include the following:

- Software implementations of cryptographic algorithms
- An API for creating interfaces to hardware cryptographic devices for offload of computationally intensive cryptographic algorithms
- X.509 certificate support

The cryptography libraries provide support for other VxWorks 7 technologies, such as IPsec, IKE, SSH, and SSL. They can also be used for other applications requiring cryptographic processing.

**Supported Algorithms on page 1**The VxWorks 7 cryptography libraries are based upon the latest OpenSSL 1.1.1 release available at the time of the VxWorks release.
**FIPS 140-2 Security on page 1**VxWorks 7 cryptography libraries provide support for the Federal Information Processing Standard (FIPS) 140-2, Security Requirements for Cryptographic Modules.
**RFCs on page 2**
**X.509 Certificates May Require a Real-World Clock on page 2**An X.509 certificate validation may require that the correct real-world time be properly set.

## 1. Supported Algorithms

The VxWorks 7 cryptography libraries are based upon the latest OpenSSL 1.1.1 release available at the time of the VxWorks release.

The cryptography libraries include two additional algorithms that are not part of OpenSSL:

- Advanced Encryption Standard (AES) key wrap
- AES cypher-based message authentication code (CMAC)

For information about the algorithms supported by the VxWorks cryptography libraries, see the manpages accessible from the OpenSSL website: https://www.openssl.org/docs/manpages.html.

**Parent topic:** Overview on page 1

## 2. FIPS 140-2 Security

VxWorks 7 cryptography libraries provide support for the Federal Information Processing Standard (FIPS) 140-2, Security Requirements for Cryptographic Modules.

The OpenSSL FIPS module is not available with the OpenSSL 1.1.1 release. As a result, the VxWorks layer **OPENSSL_FIPS** and components **INCLUDE_OPENSSL_FIPS** and **INCLUDE_OPENSSL_FIPS_CTRL_CMD** are present but are not functional or supported. For more information, see the release notes.

**Parent topic:** Overview on page 1

# 3. RFCs

The Wind River Cryptography Libraries comply with the IETF RFCs listed in this section. The RFCs are accessible from the following website:

http://www.ietf.org

Cryptography

- RFC 1319: *The MD2 Message-Digest Algorithm*
- RFC 1320: *The MD4 Message-Digest Algorithm*
- RFC 1321: *The MD5 Message-Digest Algorithm*
- RFC 2040: *The RC5, RC5-CBC, RC5-CBC-Pad, and RC5-CTS Algorithms*
- RFC 2144: *The CAST-128 Encryption Algorithm*
- RFC 2104: *HMAC: Keyed-Hashing for Message Authentication*
- RFC 2202: *Test Cases for HMAC-MD5 and HMAC-SHA-1*
- RFC 2268: *A Description of the RC2(r) Encryption Algorithm*
- RFC 2631: *Diffie-Hellman Key Agreement Method*
- RFC 3394: *Advanced Encryption Standard (AES) Key Wrap Algorithm*
- RFC 4492: *Elliptic Curve Cryptography (ECC) Cipher Suites for Transport Layer Security (TLS)*
- RFC 4493: *The AES-CMAC Algorithm*

X.509 Certificates

- RFC 2459: *Internet X.509 Public Key Infrastructure Certificate and CRL Profile*
- RFC 2510: *Internet X.509 Public Key Infrastructure Certificate Management Protocols*
- RFC 2511: *Internet X.509 Certificate Request Message Format*

**Parent topic:** Overview on page 1

# 4. X.509 Certificates May Require a Real-World Clock

An X.509 certificate validation may require that the correct real-world time be properly set.

When a board without a real-world clock boots with VxWorks 7, the system time is set to Jan 1 1970, 00:00. An X.509 certificate is only valid for a specific time period as defined in the certificate by the **Not Before** and **Not After** validity values. This means that for the X.509 certificate validation to work it needs to have the system time set to the correct real-world time.

Use **clockLib** to set VxWorks 7 to the real-world time. See the *VxWorks POSIX Programmer's Guide: POSIX Clocks and Timers Overview* for more information regarding setting the clock.

Example X.509 certificate validity values:

```
Validity
           Not Before: Aug  1 00:00:00 2016 GMT
           Not After : Dec 31 23:59:59 2018 GMT
```

**Parent topic:** Overview on page 1



WNDRVR

# 2. VXWORKS CONFIGURATION REFERENCE

VxWorks can be configured with cryptography facilities, hardware cryptography support, a key store, and a secrets repository.

**VSB Options for Cryptography Libraries on page 3**The VSB options enable the appropriate libraries, algorithms, and hardware for cryptography libraries.
**VIP Components and Parameters for Cryptography Libraries on page 4**The VIP components allow you to add cryptography libraries, a secrets repository, the Federal Information Processing Standard (FIPS) 140-2 mode, and shell commands to your project.

## 1. VSB Options for Cryptography Libraries

The VSB options enable the appropriate libraries, algorithms, and hardware for cryptography libraries.

| VSB Layer | Description |
|---|---|
| **CRYPTOMISC** | Enables hardware encryption, IAF, IPFREESCALE, IPHWCRYPTO, and TPM libraries for TPM 2.0 devices.<br>The Freescale cryptographic hardware libraries enable hardware cryptography offload for Freescale security engines known as hardware cryptography devices.<br>Note: Configuration options for **COMPONENT_IPFREESCALE** are embedded in the **ipfreescale_config.h** file. These options must be configured before you build the VSB.<br>The VxWorks Information Assurance Framework (IAF) enables the SEC 5.x driver (polled mode) and a vault key-encryption-password backend that leverages the SEC 5.x device. |
| **OPENSSL** | Enables the encryption libraries and transport layer security (TLS) network protocol.<br>The **COMPONENT_IPCRYPTO** option is included to enable cryptography, a port of the **OPENSSL** encryption library, excluding the secure sockets layer (SSL) part which is found in **IPNET_SSL**. This option makes the following components available to the VIP:<br>• **INCLUDE_IPCRYPTO**<br>• **INCLUDE_GENRSA_CMD**<br>• **INCLUDE_DSA_CMD**<br>• **INCLUDE_IPCRYPTO_USE_TEST_CMDS** |
| **OPENSSL_FIPS** | The OpenSSL FIPS module is not available with the OpenSSL 1.1.1 release. As a result, the VxWorks layer **OPENSSL_FIPS** and components **INCLUDE_OPENSSL_FIPS** and **INCLUDE_OPENSSL_FIPS_CTRL_CMD** are present but are |

| VSB Layer | Description |
|---|---|
| | not functional or supported. For more information, see the release notes. |
| **SECURITY_MISC** | Enables encryption and protected storage. The implementation is based on OpenSSL.<br>Also enables the VxWorks API for hash algorithms, hash message authentication code (hmac) and password-based key derivation function 2 (pbkdf2).The implementation is based on OpenSSL. |
| **TPM2_TSS** | Enables the TSS 2.0 stack and makes **INCLUDE_SEC_VAULT_KEP_TPM** and **INCLUDE_TPM2_OPENSSL_ENGINE** available. |

**Parent topic:** VxWorks Configuration Reference on page 3

# 2. VIP Components and Parameters for Cryptography Libraries

The VIP components allow you to add cryptography libraries, a secrets repository, the Federal Information Processing Standard (FIPS) 140-2 mode, and shell commands to your project.

| Components and Parameters | Description |
|---|---|
| **INCLUDE_IPCRYPTO**<br>• **OPENSSL_HOME**<br>• **OPENSSL_CONF** | Includes the cryptography libraries.<br>The **OPENSSL_HOME** parameter specifies the OpenSSL home directory.<br>The **OPENSSL_CONF** parameter specifies the OpenSSL configuration file. |
| **INCLUDE_GENRSA_CMD** | Provides the OpenSSL genrsa shell command. |
| **INCLUDE_DSA_CMD** | Provides the OpenSSL dsa shell command. |
| **INCLUDE_IPCRYPTO_USE_TEST_CMDS** | Adds shell commands to manage test methods for software-based cryptography. |
| **INCLUDE_TPM2_OPENSSL_ENGINE** | TPM2-TSS cryptography engine for OpenSSL. |
| **INCLUDE_IPFREESCALE_CMD** | Adds shell commands to manage test methods for the Freescale security engines. |
| **INCLUDE_IPHWCRYPTO** | Provides cryptography hardware support for the cryptography libraries |
| **INCLUDE_IPHWCRYPTO_USE_CMDS** | Adds shell commands to manage the operation of hardware-based cryptography. |



WNDRVR

| Components and Parameters | Description |
|---|---|
| **INCLUDE_IPHWCRYPTO_USE_TEST_CMDS** | Adds shell commands to manage test methods for hardware-based cryptography. |
| **INCLUDE_OPENSSL_FIPS**<br>• **FIPS_SIGNATURE** | The OpenSSL FIPS module is not available with the OpenSSL 1.1.1 release. As a result, the VxWorks layer **OPENSSL_FIPS** and components **INCLUDE_OPENSSL_FIPS** and **INCLUDE_OPENSSL_FIPS_CTRL_CMD** are present but are not functional or supported. For more information, see the release notes. |
| **INCLUDE_OPENSSL_FIPS_CTRL_CMD** | The OpenSSL FIPS module is not available with the OpenSSL 1.1.1 release. As a result, the VxWorks layer **OPENSSL_FIPS** and components **INCLUDE_OPENSSL_FIPS** and **INCLUDE_OPENSSL_FIPS_CTRL_CMD** are present but are not functional or supported. For more information, see the release notes. |
| **INCLUDE_SEC_KEY_STORE** | Provides storage protection for OpenSSL EVP_PKEY keys. |
| **INCLUDE_SEC_KEY_STORE_CMD** | Shell command for administration of OpenSSL EVP_PKEY keys in the key store. |
| **INCLUDE_SEC_SECRET**<br>• **SEC_VAULT_FILE_ROOT** | Provides storage protection (secrets repository) for arbitrary secrets such as passwords, encryption keys, and shared secrets.<br>The **SEC_VAULT_FILE_ROOT** parameter specifies the top-level directory where the key store and vault directory structure will be created. The subdirectories **keys** and **vault** are created automatically under this directory when keys or passwords are imported.<br>For deployed systems, **SEC_VAULT_FILE_ROOT** can be set to a directory in a persistent file system such as flash memory. For backwards compatibility, it is set to **/ram**<br>For information about implementing a temporary RAM-based secrets repository, see Secrets Repository Usage on page 13. |
| **INCLUDE_SEC_SECRET_CMD** | Adds shell commands to manage the operation of the secrets repository. |
| **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**<br>• **SEC_VAULT_KEY_ENCRYPTING_PW** | Obfuscates the key-encrypting password and stores it in the image<br>The **SEC_VAULT_KEY_ENCRYPTING_PW** parameter defines the password used to encrypt keys in the secrets repository and key store. The parameter value should be set to a random string of at least 40 characters containing both lower and upper case letters, numbers, and special characters.<br>No default is provided by intention, to ensure that unique values are used. You must define a password, otherwise your VIP will not build. |
| **INCLUDE_SEC_VAULT_KEP_TPM** | Seals the key-encrypting password by trusted platform module trusted platform module 2.0 (TPM 2.0) hardware and unseals it |

| Components and Parameters | Description |
|---|---|
| • **DRV_TPM_INSTANCE_DEV_NAME** | when the secrets repository or secure key store module is accessed.<br>The **DRV_TPM_INSTANCE_DEV_NAME** parameter specifies the I/O system name for the TPM device, which is also used by the TPM driver. The default name is **/tpm0**.<br>If you use TPM device storage, you must also select the appropriate TPM driver for your system (for example, DRV_IA_TPM). |
| INCLUDE_IPCOM_SHELL_CMD | Include the IPCOM shell command interface to access the fips_ctrl shell command provided by the **INCLUDE_OPENSSL_FIPS_CTRL_CMD** component. |

**Parent topic:** VxWorks Configuration Reference on page 3

# 3. KEY STORE AND SECRETS REPOSITORY

The key store stores public key pairs. The key pairs that can be used include Rivest Shamir Adleman (RSA), digital signature algorithm (DSA), and elliptic curve cryptography (ECC) keys. The secrets repository stores passwords and symmetric encryption keys.

The key store is used for user passwords (in clear text) that are used by applications to log users onto other systems.

The public key pairs, and the passwords and symmetric encryption keys, are stored encrypted in files. A key-encrypting password (KEP) is used to generate the encryption key. The same key-encrypting password that is used for the secrets repository is also used for the key store. The key-encrypting is password produced by a KEP provider (for which there are several options).

The user defines the base directory for the key pairs and passwords when the system is configured. The sub-directories **keys** and **vault** are automatically created below it when keys or passwords are added. For example:

```
/sd0:1/keys/mySslKey
/sd0:1/vault/myPw
```

The key store and secrets repository provide APIs and shell command for working with those facilities, such as for creating, importing, and deleting keys.

Note: The key store and secrets repository replaced the legacy secure key database. The legacy secure key database is supported for backwards compatibility reasons, but is deprecated. The secure key database APIs are now mapped to the new key store and secrets repository APIs. This means that a particular key ID can be accessed via both the key store and secrets repository, and secure key database APIs.

**Key-Encrypting Password Providers on page 7**There are several options for key-encrypting password providers.
**Key Store APIs and Shell Command on page 9**The key store provides APIs and a shell command for key management.
**Secrets Repository APIs and Shell Commands on page 10**The secrets repository provides APIs and a shell command for data management.
**Key Store Usage on page 11**Usually it is a system administrator who generates a key pair on the target or the host system, and then adds them to the VxWorks key store with an import function or shell command. Application code makes use of the keys by way of their key store ID.
**Secrets Repository Usage on page 13**Arbitrary secrets such as passwords, encryption keys, and shared secrets can be added to the VxWorks secrets repository with an import function or shell command. Application code makes use of the secrets by way of their secrets ID.

## 1. Key-Encrypting Password Providers

There are several options for key-encrypting password providers.

The key-encrypting password (KEP) used to encrypt the keys in the key store, and the secrets in the secrets repository, is derived from a text password. This password string is used as input to the PKCS#5 for the generation of an encryption key and initialization vector. The KEP is returned by the provider for both get and import operations. It is needed to derive the key for encryption (import) and decryption (get).

The following KEP providers may be used (one per system):

- password obfuscator KEP provider
- TPM 2.0 KEP provider
- a custom KEP provider

## Password Obfuscator KEP Provider

A password obfuscator transforms a password string into a new string used as input to the PKCS#5 for the generation of an encryption key and initialization vector. The main protection that the password obfuscator provides is that an attacker cannot simply run the strings command on the image to get a list of passwords decrypt the files in the key store and secrets repository.

Using this KEP provider requires:

- VSB configuration with the **SECURITY_MISC** layer.
- VIP configuration with the **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR** component.
- Setting the **SEC_VAULT_KEY_ENCRYPTING_PW** parameter to a random-string password.

  The random string should consist of at least 40 characters containing both lower and upper case letters, numbers, and special characters. No default is provided by intention, to ensure that unique values are used. You must define a password, otherwise your VIP will not build.

## TPM 2.0 KEP Provider

The TPM 2.0 KEP provider seals a password string with TPM 2.0 hardware. If a key-encrypting password is not already stored in the TPM device, the KEP provider automatically generates a random string for the password and stores it.

Using this KEP provider requires:

- VSB configuration with the **SECURITY_MISC** and **TPM2_TSS** layers.
- VIP configuration with the **INCLUDE_SEC_VAULT_KEP_TPM** component.
- Setting the **DRV_TPM_INSTANCE_DEV_NAME** parameter to the I/O system name for the TPM device.

If your boot loader (such as UEFI) does not support TPM, you will need to use the password obfuscator password provider for the boot loader itself.

## BLOB KEP Provider for NXP 5.x Security Engine

NXP 5*x* security engines protect data in a cryptographic data structure called a binary large object (BLOB), which provides both confidentiality and integrity protection. NXP 5*x* security engines are included, for example, with the T1040 and T2080 processors.

The security engine provides a method for protecting user-defined data across system power cycles. The data is encrypted so that it can be safely stored non-volatile storage before the chip is powered down. Each time the BLOB protocol is used to protect data, a different randomly-generated key is used to encrypt the data. This key is itself encrypted using a key encryption key, and the resulting encrypted key is then stored along with the encrypted data. The key-encryption key is derived from the chip's master secret key, so it can be recreated when the chip is powered up again.

Using this KEP provider requires:

- VSB configuration with the **CRYPTOMISC** layer.
- VIP configuration with **INCLUDE_IAF** and **INCLUDE_SEC_VAULT_KEP_SEC**.
- Setting the **SEC_KEP_BLOB_FILENAME** parameter to the KEP BLOB file name. It is stored at the same level as the **keys** and **vault** sub-directories.
- Setting the **SEC_VAULT_FILE_ROOT** parameter to the name of the directory holding the **keys** and **vault** sub-directories.

Then when the target is booted, the secKepSet( ) function must be called to set the key encrypting password. It uses the filename set with the **SEC_KEP_BLOB_FILENAME** configuration parameter, and the directory name set with the **SEC_VAULT_FILE_ROOT** parameter. If the call is successful, the KEP data is encrypted into an encrypted BLOB by the SEC 5.x engine using its key, and stored in a file called **kep.blob**, in the vault directory defined by **SEC_VAULT_FILE_ROOT**. If the KEP blob file already exists, the secKepSet( ) call fails.

**CAUTION:** **Do not hard-code the key-encryption password in the call to secKepSet( ). This would be unsafe as the key-encryption password would be included in the image in clear text.**

### Custom KEP Provider

Users can install a custom KEP provider using the secVaultSetKepCb( ) function.

**Parent topic:** Key Store and Secrets Repository on page 7

# 2. Key Store APIs and Shell Command

The key store provides APIs and a shell command for key management.

### Key Store APIs

secKeyStoreGet( )

```
EVP_PKEY* secKeyStoreGet(const char* keyId);
```

Returns the key pair with the specified **keyId**. The returned **EVP_PKEY** must be freed by the caller by calling EVP_PKEY_free( ).

secKeyStoreX509Get( )

```
EVP_PKEY* secKeyStoreX509Get(X509* x509, char* keyId, int keyIdLen);
```

Returns the key pair matching the public key in the X509 certificate given as input argument. **keyId** and **keyIdLen** are output parameters. **keyId** is a pointer to a buffer where the **keyId** of the key pair will be stored. **keyIdLen** holds the length of the **keyId** buffer. The returned **EVP_PKEY** must be freed by the caller by calling EVP_PKEY_free( ).

secKeyStoreImport( )

```
STATUS secKeyStoreImport(const char* keyId, const EVP_PKEY* key);
```

Imports **key** into the key store and gives it key ID **keyId**.

secKeyStoreDelete( )

```
STATUS secKeyStoreDel(const char* keyId);
```

Deletes a key from the key store.

### Key Store Shell Command

```
[vxWorks *]# keyStore
keystore [options]
   list
   import <id> <file> <key-type> - imports the <key-type> specific key in <file> and gives it id <
id>
                                   <key-type> could be 'tpm2' or 'pkey'. If it is not specified, i
```



WNDRVR

```
t uses
                                    'pkey' by default
   delete <id> - deletes <id> from the key store
   print <id> - prints the public key of <id>
   find <cert file> - prints the id of the key matching the cert in <cert file>
   help|-h

[vxWorks *]#
```

**Parent topic:** Key Store and Secrets Repository on page 7

# 3. Secrets Repository APIs and Shell Commands

The secrets repository provides APIs and a shell command for data management.

## Secrets Repository APIs

secSecretImport( )

```
STATUS secSecretImport(const char* id, const unsigned char* secret, int len);
```

Imports data into the secrets repository. Data can be any sensitive data. Typical examples would be passwords that must be available in clear text, encryption keys, IKE pre-shared keys, WiFi passwords, etc.

The **id** argument holds the identity, or name, of the data to import. The **secret** argument points to the data that is to be imported. The **len** argument holds the number of bytes to import.

secSecretGet( )

```
int secSecretGet(const char* id, unsigned char* secret, int* secretLen);
```

Get data from the secrets repository. The data stored as **id** is written to the data buffer. The **secret** argument points to the data. The **secretLen** argument holds the length of the data buffer.

secSecretDel( )

```
STATUS secSecretDel(const char* id);
```

Delete data from the secrets repository. Data that is deleted from the secrets repository cannot be restored.

## Secrets Repository Shell Command

```
[vxWorks *]# secret
secret [options]:
        list - list all secret's id
        import <id> [secret] - import <secret> into vault and give it id <id>
        delete <id> - delete <id> from the vault
        verify <id> [secret] - verify <id> with <secret>
        help|-h
```



WNDRVR

```
[vxWorks *]#
```

**Parent topic:** Key Store and Secrets Repository on page 7

# 4. Key Store Usage

Usually it is a system administrator who generates a key pair on the target or the host system, and then adds them to the VxWorks key store with an import function or shell command. Application code makes use of the keys by way of their key store ID.

## VxWorks Configuration

The VSB must include the **OPENSSL** and **SECURITY_MISC** layers. If you want to use key store to manage the key pairs generated by the TPM2 OpenSSL engine, you must add the **TPM2_TSS** layer to your VSB.

The VIP must include:

- **INCLUDE_IPCRYPTO**
- **INCLUDE_SEC_KEY_STORE**

> Note: When you add this component, you will get a configuration error, because it automatically adds **INCLUDE_SEC_VAULT_KEP_OBFUSCATOR**, and its parameter **SEC_VAULT_KEY_ENCRYPTING_PW** must be set. For security reasons, the parameter is not set to a default value.

- **INCLUDE_SEC_KEY_STORE_CMD** (for key store management commands)
- **INCLUDE_GENRSA_CMD** (for the genrsa key generation command)
- **INCLUDE_DSA_CMD** (for the dsa key generation command)
- **INCLUDE_SEC_SECRET**
- **SEC_VAULT_FILE_ROOT** and set it to the base directory for the key store and secrets repository
- **INCLUDE_TPM2_OPENSSL_ENGINE** is required if you want the key store to manage the key pairs generated by the TPM2 OpenSSL engine.
- A KEP provider (see Key-Encrypting Password Providers on page 7)
- A storage system.

## Creating Keys

RSA keys and DSA keys can be generated from the VxWorks kernel shell (with the command interpreter) using the genrsa and dsa commands. For example:

```
[vxWorks *]# genrsa 2048 -out my-rsa-key-pair.pem
```

RSA keys can also be generated from the TPM2 OpenSSL engine if the layer **TPM2_TSS** is included in the VSB. For example:

```
[vxWorks *]# tpm2tss-genkey.vxe -a rsa -s 2048 my-tpm2-rsa-key-pair.pem --tcti=libtpm20tss.so.1:/tpm0
```

The syntax for these commands can be displayed using the --help option.



WNDRVR

## Importing Keys

Key pairs can be imported using the secKeyStoreImport( ) function or the keystore command. In either case, a *keyID* is assigned to the key pair when it is imported. The *keyID* is used by any application that needs to access the key pair in the store. VxWorks applications such as SSH must be configured to use the same *keyID* as is used when the key pair is imported.

As an example, assume that an application listens to incoming SSL connections. This application needs to have an X509 certificate and a matching private/public key pair. The X509 certification does not need to be protected against unauthorized reading. It can simply be stored in the file system. The key pair, however, is sensitive and should be stored in the key store.

Using the keystore shell command to import the key, for example:

```
[vxWorks *]# keyStore import mySslKey mySslKey.pem
```

This command imports the key pair and assigns them the ID **mySslKey**. The ID then identifies the key pair in the key store. For example:

```
[vxWorks *]# keyStore list
mySslKey
[vxWorks *]# ls "/ram0/keys"
/ram0/keys/.
/ram0/keys/..
/ram0/keys/mySslKey
```

**CAUTION:** **For security reasons, the system administrator should now delete mySslKey.pem from the file system.**

Note: In this example, the system has been configured with a temporary key store in RAM, which can be useful for the development and test process. The system was configured with **INCLUDE_RAM_DISK** and **INCLUDE_RAM_DISK_FORMAT_ANY**. The value of **SEC_VAULT_FILE_ROOT** has been set to **"/ram0"**. For a production system, you would use a persistent file system.

## Runtime Use

At runtime, the application can then fetch the SSL key from the key store using the ID, with code like this:

```
EVP_PKEY* sslKey;

sslKey = keyStoreGet("mySslKey");
if (!sslKey)
   /*  error handling */

/* use the sslKey as appropriate for your application */

/* Once done with the key, free it */
EVP_PKEY_free(sslKey);
```

**Parent topic:** Key Store and Secrets Repository on page 7

# 5. Secrets Repository Usage

Arbitrary secrets such as passwords, encryption keys, and shared secrets can be added to the VxWorks secrets repository with an import function or shell command. Application code makes use of the secrets by way of their secrets ID.

## VxWorks Configuration

The VSB must include the **OPENSSL** layer.

The VIP must include:

- **INCLUDE_IPCRYPTO**
- **INCLUDE_SEC_SECRET**
- **SEC_VAULT_FILE_ROOT** set to the base directory for the key store and secrets repository
- **INCLUDE_SEC_SECRET_CMD** (for secrets repository management commands)
- A KEP provider (see Key-Encrypting Password Providers on page 7)
- A storage system.

## Importing Secrets

At runtime, use the secSecretImport( ) function, or the secret import shell command (using the command interpreter), to store secrets in the secrets repository. For example:

```
[vxWorks *]# secret import myPw myPassWordFile
```

This command imports the password and assigns it the ID **myPw**. The ID then identifies the password in the secrets repository. For example:

```
[vxWorks *]# secret list
myPw
[vxWorks *]# ls "/ram0/vault"
/ram0/vault/.
/ram0/vault/..
/ram0/vault/myPw
```

The secret ID can be used by any application that needs to access the password.

Note: In this example, the system has been configured with a temporary key store in RAM, which can be useful for the development and test process. The system was configured with **INCLUDE_RAM_DISK** and **INCLUDE_RAM_DISK_FORMAT_ANY**. The the value of **SEC_VAULT_FILE_ROOT** has been set to **"/ram0"**. For a production system, you would use a persistent file system.

**Parent topic:** Key Store and Secrets Repository on page 7

# 4. TRUSTED KEY STORE

VxWorks can be configured with a trusted key store for public key certificates. The trusted key store is then used to determine whether or not the owner of the public key is trusted. Signature verification is done by the entity calling the trusted key store.

The trusted key store is used, for example, with the secure ELF loader facility to validate the private signatures of kernel applications, RTP applications, shared libraries, plug-ins, and the system symbol table (when separate from VxWorks) before they are loaded.

**Implementing a Static Trusted Key Store on page 14**Add OpenSSL and cryptography support to your VSB. Add the trusted key store component to your VIP. Configure the VIP with your X509 certificate. The public key certificate is stored in the VxWorks image.
**UEFI Key Database Trusted Key Store on page 15**The UEFI key database can be used for a trusted key store. This feature can only be used with the secure UEFI boot loader, and not with a standard UEFI boot loader.
**i.MX6 Trusted Key Store on page 15**The trusted public key for the i.MX6 is stored in a super root key (SRK). The SRK is an RSA key pair (private and public). It is used to sign the image. The SRK hash is stored in an i.MX6 eFUSE (one-time programmable non-volatile memory).
**Custom Trusted Key Store Provider on page 16**You can implement a custom trusted key store provider for the certificate store.

## 1. Implementing a Static Trusted Key Store

Add OpenSSL and cryptography support to your VSB. Add the trusted key store component to your VIP. Configure the VIP with your X509 certificate. The public key certificate is stored in the VxWorks image.

### About This Task

### About This Task

The trusted public key is added to the user configlette, **staticTrustStoreCfg.c**, which is copied to the VIP project. The public key is configured by adding a PEM-encoded X509 certificate in the configlette. The X509 certificate is only used as a container for the public key. No certificate validation is done by the static trusted key store itself (no checking of subject name, validity times, signature, and so forth).

### Procedure

1. Create a VSB and add the **OPENSSL** and **SEC_CRYPTO** layers.
2. Build the project.
3. Create a VIP based on the VSB.
4. Add the **INCLUDE_STATIC_TRUST_STORE** component.
5. Save your project.

   The **staticTrustStoreCfg.c** file is added to your project.

6. Edit **staticTrustStoreCfg.c** to add your X509 certificate to the configuration.
7. Build the project.

   The VxWorks image includes the public key certificate.

**Parent topic:** Trusted Key Store on page 14

## 2. UEFI Key Database Trusted Key Store

The UEFI key database can be used for a trusted key store. This feature can only be used with the secure UEFI boot loader, and not with a standard UEFI boot loader.

VxWorks can be configured with support for using the UEFI database as a trusted key store. The PK, KEK and db EFI signature lists that are produced by the VSB build must be configured into the UEFI firmware. They are required for the UEFI firmware to authenticate the UEFI loader. The UEFI key database trusted key store enables the secure ELF loader to use the UEFI signature list (the white list) to determine whether a key used to sign an ELF object is trusted or not. From a system administrator's point of view, this provides a single point of configuration. The key (or keys) that are used to sign all software, including the UEFI loader, VxWorks kernel image, ELF objects (DKMs, RTPs and shared libraries), is configured in the UEFI key db.

To use the UEFI database for a trusted key store:

- The VxWorks VSB project requires the **SECURE_LOADER** layer.
- The VIP project requires the **INCLUDE_UEFI_DB_TRUST_STORE** and **INCLUDE_BOOT_LOADER** components.
- The **UEFI_DB_SIZE** parameter must be set to the required size of the UEFI database section. The default value of ((4 * 1000) + 24) allows storage of 4 average-sized certificates. Note that the 24 bytes are reserved for system use.
- As part of your UEFI firmware configuration, the **PK.sig**, **KEK.sig**, and **db.sig** must be added. The files are are created in the VSB output directory *vsbDir* **/secureLoader**.

**Parent topic:** Trusted Key Store on page 14

## 3. i.MX6 Trusted Key Store

The trusted public key for the i.MX6 is stored in a super root key (SRK). The SRK is an RSA key pair (private and public). It is used to sign the image. The SRK hash is stored in an i.MX6 eFUSE (one-time programmable non-volatile memory).

Note: The i.MX6 trusted key store can only be used when the i.MX6 High Assurance Boot (HAB) is configured for fast authentication. If the HAB is configured for normal authentication, the static trusted key store provider must be used, in which case the SRK key is used to sign the RSA key that is used to sign the image. The public IMG key needs to be put into the static trusted key provider in this configuration.

To use an I.MX6 trusted key store:

- The VxWorks VSB project requires the OPENSSL and SECURITY_MISC layers.
- The VIP project requires the **INCLUDE_IMX_TRUST_STORE** component.
- You must copy the **SRK_1_2_3_4_table.bin** file (which was generated with your secure boot support VIP project) to the trusted key store VIP directory, and change the file name to **SRK_table.bin**.

**Parent topic:** Trusted Key Store on page 14



WNDRVR

# 4. Custom Trusted Key Store Provider

You can implement a custom trusted key store provider for the certificate store.

To do so, you must do the following:

- Implement this API:

```
STATUS secTrustStorePubKeyValidate(const char* trustStoreId, const X509* cert, const unsigned
int keyUsage);
```

- Create a CDF component and add it to **SELECT_TRUST_STORE_PROVIDER**. The **INIT_RTN** of the new component must call secTrustStorePubKeyValidateProviderSet( ) to register the new provider.

The static trusted key store and the trusted key store for i.MX6 are examples of trusted key stores. They are implemented using the same API as you would use with a custom trusted key store provider.

**Parent topic:** Trusted Key Store on page 14

# 5. APIS FOR CRYPTOGRAPHY LIBRARIES

The VxWorks cryptography libraries provide the following APIs:

| API | Description |
|---|---|
| **ipcrypto** | Implements the advanced encryption standard (AES) cipher-based message authentication code (CMAC) and AES key wrap algorithms. |
| **ipfreescale** | Implements a cryptographic engine for Freescale processors. |
| **iphwcrypto** | Implements cryptographic hardware support. |
| **secCipher** | Implements symmetrical encryption. |
| **secHash** | Enables the calculation of a hash digest. |
| **secHmac** | Enables the calculation of a hash message authentication code (hmac). |
| **secKeyStoreLib** | Enables the storage of key pairs used for SSL connections. |
| **secPbkdf2** | Implements the public-key cryptography standards 5 (PKCS5) password-based key derivation function 2.0. |
| **secSecretLib** | Enables the storage of symmetric encryption keys and passwords. |

For more information on the APIs and their functions, see the *VxWorks 7 Kernel Security API Reference*.

# 6. HARDWARE CRYPTOGRAPHY DEVICES

The VxWorks 7 cryptography libraries provide functions for offloading cryptographic operations onto hardware cryptography devices.

This provides an effective way to improve the performance of applications requiring computationally intensive cryptography support.

The cryptography libraries use the concept of an *engine*, which corresponds to a hardware cryptography device. Depending on the capabilities of the hardware, the engine can handle multiple types of cryptographic operations.

Essentially, an engine is a set of pointers to callback functions, where each callback function handles hardware offload for a given algorithm. For information on the APIs and their functions, see the **iphwcrypto** and **ipfreescale** APIs in the VxWorks 7 Kernel Security API Reference.

To make use of an engine in an application, add the engine using iphwcrypto_engine_add( ). Then, add any of the supported algorithms using the **iphwcrypto_engine_add**_*xxx*() and **iphwcrypto_engine_register**_*xxx*() functions.

Adding an algorithm to an engine sets internal pointers in the engine, pointing to the functions that implement hardware support for the algorithm. Registering a class of algorithms notifies the cryptography libraries that the engine should be the default method of processing those algorithms.

If hardware cryptography fails, the cryptography libraries fall back to the software implementation. This option can be turned off as part of the VxWorks source build. To turn off this software fallback option, edit **iphwcrypto_config.h** to remove the definition of **IPHWCRYPTO_REVERT_FAILURES_TO_SOFTWARE**.

Other than adding the engine and adding and registering algorithms, you do not need to change your application to use cryptographic hardware. All requests for cryptographic services are the same whether the cryptography libraries use the software implementation or the hardware implementation of an algorithm.

## Cryptography Libraries Engines and OpenSSL Engines

A cryptography libraries engine is a data structure that keeps track of the algorithms that should be processed by the cryptographic hardware. Primarily, it contains function pointers to the functions that handle the algorithms.

The key interface in OpenSSL is the ENGINE API. Internally, the cryptography libraries engine uses the OpenSSL ENGINE API. However, do not call any of the OpenSSL ENGINE functions directly. Make all access to the cryptography libraries engines using the cryptography libraries functions provided. The cryptography libraries support a single engine in an application which represents a single hardware device.

## Data Types

Public data types for the hardware functions of the cryptography libraries are defined in the following file:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-**x.x.x.x**/iphwcrypto/h/ iphwcrypto.h**

This file includes important definitions, including function signatures for the callback functions and identifiers for the supported algorithms.

**Cryptography Libraries Application Interfaces on page 19**The cryptography libraries API includes the OpenSSL API plus additional functions such as functions to support the advanced encryption standard (AES) cipher-based message authentication code (CMAC) and AES key wrap algorithms.
**Cryptographic Engines on page 19**An engine contains function pointers to callback functions that implement the necessary interfaces between VxWorks 7 cryptography libraries and the hardware device.
**Creating a Cryptographic Engine on page 20**To create a cryptographic engine, you must write the callback functions, add the engine and algorithms to the VxWorks application, and register the algorithms.
**Message Digests on page 22**The message digest types include message-digest 5 (MD5), secure hash algorithm 1 (SHA-1), and SHA-1 with digital signature algorithm (DSA).

**Ciphers on page 24**The cipher types include data encryption standard (DES), 3DES (Triple-DES), and advanced encryption standard (AES).
**Public Key Algorithms on page 25**A cryptography libraries engine can compute modular exponents for the Diffie-Hellman, Rivest-Shamir-Adleman (RSA), and digital signature algorithm (DSA) public key algorithms.
**Random Number Generators on page 25**A cryptography libraries engine can generate random numbers using a hardware cryptographic device.
**IPsec ESP Processing on page 26**A cryptography libraries engine can work with Wind River IPsec to process IPsec encapsulating security payload (ESP) packets using a hardware cryptographic device.

# 1. Cryptography Libraries Application Interfaces

The cryptography libraries API includes the OpenSSL API plus additional functions such as functions to support the advanced encryption standard (AES) cipher-based message authentication code (CMAC) and AES key wrap algorithms.

These additional functions are provided by the **ipcrypto** API. For more information, see the VxWorks 7 Kernel Security API Reference.

For information about the OpenSSL API, see the OpenSSL website.

### Coding Examples

There are many examples of OpenSSL usage online and in printed sources. You can also refer to the Wind River source code in the SSH and IKE products for examples.

For a specific example in the Wind River source code, see the following file:

*installDir* **/vxworks-7/pkgs_v2/security/openSSL-***version* **/openssl-***version* **/crypto/evp/ evp_test.c**

**Parent topic:** Hardware Cryptography Devices on page 18

# 2. Cryptographic Engines

An engine contains function pointers to callback functions that implement the necessary interfaces between VxWorks 7 cryptography libraries and the hardware device.

Each cryptographic algorithm managed by the engine requires one or more callback functions for handling different aspects of the algorithm. Message digests, for example, require five callback functions to handle initialization, updating, finalization, copying, and cleaning up. Random number generation requires a single function that returns a random number.

Creating an engine for a new hardware device requires writing the callback functions, then hooking them into the cryptography libraries.

The callback functions must return codes indicating the status of the request. For most algorithms, the possible codes are as follows:

- 1 - success
- 0 - timeout
- -1 - error

IPsec Encapsulating Security Payload (ESP) input and output callback functions can return an additional status code:

- 2 - asynchronous request submitted

**Parent topic:** Hardware Cryptography Devices on page 18

# 3. Creating a Cryptographic Engine

To create a cryptographic engine, you must write the callback functions, add the engine and algorithms to the VxWorks application, and register the algorithms.

## About This Task

The following procedure summarizes the steps you must take to create a cryptography libraries offload engine. For information on the API functions, see the see the VxWorks 7 Kernel Security API Reference.

## Procedure

1. Write the callback functions.

   Write the necessary callback functions for the algorithms you want to implement in your engine. You can use the following header file as a prototype to begin your development:

   *installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/ipfreescale/h/ ipfreescale.h**

   Each cryptographic algorithm managed by the engine requires one or more callback functions for handling different aspects of the algorithm.

   For more information about the required callbacks, refer to the section appropriate to the algorithms you are implementing in your engine. The related sections include:

   - Message Digests on page 22
   - Ciphers on page 24
   - Public Key Algorithms on page 25
   - Random Number Generators on page 25
   - IPsec ESP Processing on page 26

   Note: Algorithms that you do not specifically add and register as part of the engine are processed in software.

2. Hook the callback functions to the cryptography libraries.

   Include the **iphwcrypto.h** header file from your engine implementation in the engine implementation and startup code. The file location is as follows:

   *installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/iphwcrypto/h/ iphwcrypto.h**

3. Add the engine.

   An engine corresponds to a hardware cryptography device. It is identified by both a name and an ID. The ID is used in calls to engine functions. The name is for informational purposes only. The engine must be added to your VxWorks application.

   To add an engine, call the iphwcrypto_engine_add( ) function with the engine ID and name.

   For example:

```
/* Add an engine to iphwcrypto */
ret = iphwcrypto_engine_add(engine_id, engine_name);
if (ret != OK)
{
   logMsg("ERROR:ipenginename_create() :: iphwcrypto_engine_add() failed
with error %d", ret);
   return ret;
}
```

This creates the engine but does not set it up to process any algorithms. To do that, you must add and register the algorithms as described in later steps.

For a specific example of the iphwcrypto_engine_add( ) function, see the following file:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/ipfreescale/src/ ipfreescale.c**

For a complete description of the function and parameters, see the **iphwcrypto** API.

4. Add the algorithms.

   To enable hardware support for an algorithm, an application must add the algorithm's callback functions to the engine, then register the algorithm class. Adding the callback functions sets function pointers in the engine, but does not affect how the cryptography libraries operate. Registering an algorithm class notifies the cryptography libraries that the algorithms in the class can be implemented with the engine.

   To add an algorithm, call the appropriate **iphwcrypto_engine_add**_*xxx* **()** function. For example:

```
/* Add digests */
ret = iphwcrypto_engine_add_digest(engine_id,
IPHWCRYPTO_ENGINE_DIGEST_MD5, sizeof(Ipenginename_digest_context),
        ipenginename_md5_digest_init,
        ipenginename_md5_digest_update,
        ipenginename_md5_digest_final,
        ipenginename_digest_copy,
        ipenginename_digest_cleanup, 0);
if (ret != OK)
{
logMsg("ERROR:ipenginename_create() :: iphwcrypto_engine_add_digest(IPHWCRYPTO_ENGINE_DIGEST_MD
5) failed
with error %d", ret);
return ret;
}
```

   Set the last parameter to **1** in the final call to iphwcrypto_engine_add_ecb_cipher( ), iphwcrypto_engine_add_cbc_cipher( ), and iphwcrypto_engine_add_digest( ). For example:

```
ret = iphwcrypto_engine_add_ecb_cipher(engine_id, IPHWCRYPTO_ENGINE_CIPHER_AES192_ECB,
ipfreescale_aes192_ecb_cipher, 0);
if (ret != OK)
{
logMsg("ERROR:ipenginename_create() ::
iphwcrypto_engine_add_cipher(IPHWCRYPTO_ENGINE_CIPHER_AES192_ECB) failed with error %d",
return ret);
}
ret = iphwcrypto_engine_add_cbc_cipher(engine_id, IPHWCRYPTO_ENGINE_CIPHER_AES256_CBC,
ipfreescale_aes256_cbc_cipher, 1);
```



WNDRVR

```
if (ret != OK)
{
logMsg("ERROR:ipenginename_create() ::
iphwcrypto_engine_add_cipher(IPHWCRYPTO_ENGINE_CIPHER_AES256_CBC) failed with error %d",
return ret);
}
```

For specific examples of the **iphwcrypto_engine_add**_xxx**()** functions, see the following file:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-**x.x.x.x **/ipfreescale/src/ ipfreescale.c**

5. Register the algorithms.

   To register an algorithm class, call the appropriate **iphwcrypto_engine_register**_xxx**()** function. For example:

```
/* Register random method */
ret = iphwcrypto_engine_register_random(engine_id);
    if (ret != OK)
{
logMsg("ERROR:ipenginename_create() :: iphwcrypto_engine_register_random()
failed with error %d",
return ret);
}
```

   For a specific example of the **iphwcrypto_engine_register**_xxx**()** function, see the following file:

   *installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-**x.x.x.x **/ipfreescale/src/ ipfreescale.c**

6. Re-build the VSB.
7. Re-build the VIP.

**Parent topic:** Hardware Cryptography Devices on page 18

# 4. Message Digests

The message digest types include message-digest 5 (MD5), secure hash algorithm 1 (SHA-1), and SHA-1 with digital signature algorithm (DSA).

The message digest identifiers are as follows:

| Message Digest | Identifier |
|---|---|
| MD5 | IPHWCRYPTO_ENGINE_DIGEST_MD5 |
| SHA-1 | IPHWCRYPTO_ENGINE_DIGEST_SHA1 |
| SHA-1 with DSA | IPHWCRYPTO_ENGINE_DIGEST_SHA1_DSA |

MD5 and SHA-1 support RSA certificates. You need to add the SHA-1 with DSA digest if you intend to use hardware cryptography for DSA certificate signing and verification. Use the same callback functions that are used for SHA-1.



WNDRVR

Each digest type requires callback functions for the following operations:

Initialization

The digest initialization callback function must initialize the context to an empty state. It accepts a single parameter and a pointer to the context. The context is allocated internally by the cryptography libraries before the initialization function is called.

Updating

The digest updating callback function must add new data to the digest. It accepts three parameters: the context, the new data to add to the digest, and the length of the new data. The updating function can be called multiple times.

Finalization

The digest finalization callback function must store the final message digest in a buffer passed by the calling function.

Copying

The digest copying callback function copies a context to a new context. The two contexts must be independent; updating one context with new data cannot affect the other context.

Clean up

The digest clean up callback function cleans up the context and frees any dynamically allocated memory used by it.

The copying and clean up operations are optional. If they are not used, they pass null pointers. The message digest callback functions are defined in **iphwcrypto.h**.

A message digest context saves the state of the hash function over successive calls to the update function. The exact details are implementation-dependent. For example, a context could manage the entire data stream for the hash, as in the Freescale engine described in Freescale SEC Engine on page 30. Alternatively, if the hardware supports it, the context could hold intermediate hash values computed by calls to the digest update function.

Add each of the supported message digest types to the engine by calling iphwcrypto_engine_add_digest( ). On the final call, set the last parameter to 1.

Call iphwcrypto_engine_register_digests( ) to register the digests with the cryptography libraries and make the engine the default method for processing the digest types.

For example, the following code adds functions for MD5 and SHA-1 digests to an engine:

```
iphwcrypto_engine_add_digest( engine_id, IPHWCRYPTO_ENGINE_DIGEST_MD5,
    sizeof (md5_digest_context), md5_init, md5_update, md5_final,
    md5_copy, md5_cleanup, 0);
iphwcrypto_engine_add_digest( engine_id, IPHWCRYPTO_ENGINE_DIGEST_SHA1,
    sizeof (sha1_digest_context), sha1_init, sha1_update, sha1_final,
    sha1_copy, sha1_cleanup, 1);
iphwcrypto_engine_register_digests( engine_id );
```

The functions must match the callback function types. For example, md5_update( ) could be defined as follows:

```
md5_update
    (
    void *ctx,
    const unsigned char *data,
    unsigned int length
    );
```

You only need to call iphwcrypto_engine_register_digests( ) once, after all message digest types have been added.



WNDRVR

**Parent topic:** Hardware Cryptography Devices on page 18

# 5. Ciphers

The cipher types include data encryption standard (DES), 3DES (Triple-DES), and advanced encryption standard (AES).

The following table lists the supported cipher-block chaining (CBC) and electronic codebook (ECB) cipher types and their corresponding identifiers.

Each cipher type requires its own callback function, which must be added to the engine. The callback function ciphers the input data using the appropriate algorithm.

Table 1. Cipher Types and Identifiers

| Cipher | Identifier |
|---|---|
| DES CBC | **IPHWCRYPTO_ENGINE_CIPHER_DES_CBC** |
| DES ECB | **IPHWCRYPTO_ENGINE_CIPHER_DES_ECB** |
| 3DES CBC | **IPHWCRYPTO_ENGINE_CIPHER_3DES_CBC** |
| 3DES ECB | **IPHWCRYPTO_ENGINE_CIPHER_3DES_ECB** |
| AES128 CBC | **IPHWCRYPTO_ENGINE_CIPHER_AES128_CBC** |
| AES128 ECB | **IPHWCRYPTO_ENGINE_CIPHER_AES128_ECB** |
| AES192 CBC | **IPHWCRYPTO_ENGINE_CIPHER_AES192_CBC** |
| AES192 ECB | **IPHWCRYPTO_ENGINE_CIPHER_AES192_ECB** |
| AES256 CBC | **IPHWCRYPTO_ENGINE_CIPHER_AES256_CBC** |
| AES256 ECB | **IPHWCRYPTO_ENGINE_CIPHER_AES256_ECB** |

The definition of the callback function for CBC mode ciphers is different from the definition for ECB mode ciphers because the CBC mode ciphers require an initialization vector. The cipher callback function definitions (***_calc**) are in **iphwcrypto.h**.

The callback functions must be capable of both encryption and decryption. If the **enc** parameter is 1, the data from the **in** buffer is encrypted and placed in the **out** buffer. If **enc** is 0, the data from the **in** buffer is decrypted and placed in the **out** buffer. Both buffers must be the same length.

Add each of the supported cipher types to the engine by calling the appropriate function for the cipher type. The available functions are:

- iphwcrypto_engine_add_cbc_cipher( )
- iphwcrypto_engine_add_ecb_cipher( )

For example, if the callback function for an AES128 CBC cipher is aes_128_cbc( ), the following would add the cipher to the engine:

```
iphwcrypto_engine_add_cbc_cipher( engine_id,
    IPHWCRYPTO_ENGINE_CIPHER_AES128_CBC, aes_128_cbc, 0);
```

For the final call to iphwcrypto_engine_add_cbc_cipher( ) and iphwcrypto_engine_add_ebc_cipher( ), set the last parameter to 1.

Once all the cipher types have been added, call iphwcrypto_engine_register_ciphers( ) to register the ciphers with the cryptography libraries and make the engine the default method for processing the ciphers.

**Parent topic:** Hardware Cryptography Devices on page 18

# 6. Public Key Algorithms

A cryptography libraries engine can compute modular exponents for the Diffie-Hellman, Rivest-Shamir-Adleman (RSA), and digital signature algorithm (DSA) public key algorithms.

For all three algorithms, the definition of the **Iphwcrypto_mod_exp** callback function for the modular exponentiation is in **iphwcrypto.h**.

The modlen parameter is the length of the modulus parameter mod. It is also used as the length of the base, e, and out parameters. Where necessary, input parameters are padded with leading zeroes to the proper length.

If the result is shorter than modlen, the callback must pad the result with leading zeros to the proper length.

To add and register hardware support for Diffie-Hellman calculation, call the following functions:

- iphwcrypto_engine_add_dh( )
- iphwcrypto_engine_register_dh( )

To add and register hardware support for RSA calculation, call the following functions:

- iphwcrypto_engine_add_rsa( )
- iphwcrypto_engine_register_rsa( )

To add and register hardware support for DSA calculation, call the following functions:

- iphwcrypto_engine_add_dsa( )
- iphwcrypto_engine_register_dsa( )

**Parent topic:** Hardware Cryptography Devices on page 18

# 7. Random Number Generators

A cryptography libraries engine can generate random numbers using a hardware cryptographic device.

The definition of the **Iphwcrypto_random_gen** callback function for the random number generator is in **iphwcrypto.h**.

The data parameter stores the generated random number. The length parameter is the length of the random number.

To add and register hardware support for random number generation, call the following functions:



WNDRVR

- iphwcrypto_engine_add_random( )
- iphwcrypto_engine_register_random( )

**Parent topic:** Hardware Cryptography Devices on page 18

# 8. IPsec ESP Processing

A cryptography libraries engine can work with Wind River IPsec to process IPsec encapsulating security payload (ESP) packets using a hardware cryptographic device.

The engine can process both input and output packets. The cryptography libraries do not process IPsec authentication header (AH) requests using cryptographic hardware, but this is a less demanding operation that can be handled using software.

The following table lists the supported ESP algorithms and their corresponding identifiers.

Table 1. ESP Algorithms and Identifiers

| Algorithm | Identifier |
|---|---|
| Null encryption, no authentication | **IPHWCRYPTO_ENGINE_ESP_NULL_NULL** |
| Null encryption, message-digest 5 (MD5) authentication | **IPHWCRYPTO_ENGINE_ESP_NULL_MD5** |
| Null encryption, Secure hash algorithm 1 (SHA-1) authentication | **IPHWCRYPTO_ENGINE_ESP_NULL_SHA1** |
| Data encryption standard (DES) cipher block chaining (CBC), no authentication | **IPHWCRYPTO_ENGINE_ESP_DES_CBC_NULL** |
| DES CBC, MD5 authentication | **IPHWCRYPTO_ENGINE_ESP_DES_CBC_MD5** |
| DES CBC, SHA-1 authentication | **IPHWCRYPTO_ENGINE_ESP_DES_CBC_SHA1** |
| 3DES CBC, no authentication | **IPHWCRYPTO_ENGINE_ESP_3DES_CBC_NULL** |
| 3DES CBC, MD5 authentication | **IPHWCRYPTO_ENGINE_ESP_3DES_CBC_MD5** |
| 3DES CBC, SHA-1 authentication | **IPHWCRYPTO_ENGINE_ESP_3DES_CBC_SHA1** |
| AES CBC, no authentication | **IPHWCRYPTO_ENGINE_ESP_AES_CBC_NULL** |
| AES CBC, MD5 authentication | **IPHWCRYPTO_ENGINE_ESP_AES_CBC_MD5** |
| AES CBC, SHA-1 authentication | **IPHWCRYPTO_ENGINE_ESP_AES_CBC_SHA1** |
| CAST CBC, no authentication | **IPHWCRYPTO_ENGINE_ESP_CAST_CBC_NULL** |

| Algorithm | Identifier |
|---|---|
| CAST CBC, MD5 authentication | **IPHWCRYPTO_ENGINE_ESP_CAST_CBC_MD5** |
| CAST CBC, SHA-1 authentication | **IPHWCRYPTO_ENGINE_ESP_CAST_CBC_SHA1** |
| Blowfish CBC, no authentication | **IPHWCRYPTO_ENGINE_ESP_BLF_CBC_NULL** |
| Blowfish CBC, MD5 authentication | **IPHWCRYPTO_ENGINE_ESP_BLF_CBC_MD5** |
| Blowfish CBC, SHA-1 authentication | **IPHWCRYPTO_ENGINE_ESP_BLF_CBC_SHA1** |

To add hardware support for ESP processing of a given algorithm, call iphwcrypto_ipsec_engine_add_esp( ) with function pointers to the callback functions for input and output processing. If a function pointer to either function is null, the cryptography libraries process the corresponding operation using software instead of hardware.

The definition of the **Iphwcrypto_ipsec_esp** callback function for ESP processing is in **iphwcrypto.h**.

The tunnel descriptor block (tdb) contains information required for processing the ESP operation. This is a subset of the tunnel descriptor block required by Wind River IPsec. The definition for the **Iphwcrypto_esp_cbc_tdb_struct tdb** structure is:

```
typedef struct Iphwcrypto_esp_cbc_tdb_struct
{
    int             alg;
    unsigned char  *data;
    int             hdrlen;
    unsigned char  *akey;
    int             akeylen;
    int             hashlen;
    int             authlen;
    unsigned char  *ekey;
    int             ekeylen;
    unsigned char  *iv;
    int             ivlen;
} Iphwcrypto_esp_cbc_tdb;
```

Structure members of the tunnel descriptor block are as follows:

| Structure Member | Description |
|---|---|
| **alg** | The ESP algorithm to use. |
| **data** | Points to the ESP portion of the packet data, including ESP headers. |
| **hdrlen** | The header length, which provides an offset to the start of the encrypted data. |
| **akey** | Points to the authentication key. |
| **akeylen** | The length of the authentication key. |

| Structure Member | Description |
|---|---|
| **hashlen** | The message digest length. |
| **authlen** | The number of bytes in the authentication header. |
| **ekey** | Points to the encryption key. |
| **ekylen** | The length of the encryption key. |
| **iv** | Points to the initialization vector. |
| **ivlen** | The length of the initialization vector. |

Note:

The *data* parameter and the **data** member of the tdb parameter both point to the same location inside the packet (*pkt* parameter). This is the start of the ESP header.

The location of the encrypted portion of the packet is offset *tdb* **->hdrlen** bytes from the start of the ESP header. When processing ESP input packets, the callback function must place the decrypted data back into the packet at this location. When processing ESP output packets, the callback function must place the encrypted data back into the packet at this location.

## Authentication

Authenticated IPsec ESP packets store a message digest at the end of the packet. Wind River IPsec uses this digest to authenticate the packet. If the security association specifies an authentication method, the callback function must place a copy of the calculated message digest in the buffer pointed to by the *digest* parameter.

## Processing Modes

Depending on the capabilities of the hardware, ESP requests can be processed synchronously or asynchronously. Synchronous operation is simpler, but program execution blocks while the request is being processed. With asynchronous operation, the request is queued on the hardware and the function returns immediately. When the hardware has completed the request it calls another callback function that puts the processed packet onto the network stack. This generally results in better network performance.

When Wind River IPsec receives an IPsec packet, it calls the appropriate ESP input or output callback function. Synchronous callbacks return when the processing has been completed, and Wind River IPsec handles putting the packet back on the network stack. Asynchronous callbacks must register another callback function that executes after the hardware has completed the operation. This second callback must handle putting the packet back on the network stack.

Synchronous Operation

For a synchronous input operation, the ESP callback function must perform the following steps:

1. Prepare the hardware request.
2. Send the request to the hardware and wait for completion.
3. If the request completes successfully, replace the encrypted payload in the *data* buffer with the decrypted data. Value *tdb* **->hdrlen** contains the offset in the buffer for the decrypted data. That is, *data* + *tdb* **->hdrlen** points to the location for the data.
4. If the packet is authenticated, store a copy of the message digest in the *digest* buffer.
5. Return one of the following status codes:
   - 1 - success

- 0 - timeout
- -1 - error

An output operation is similar, except that it places encrypted data into the *data* buffer.

When the ESP callback returns successfully, IPsec puts the processed data back onto the network stack.

Asynchronous Operation

For an asynchronous input operation, the ESP callback function must perform the following operations:

1. Prepare the hardware request. This must include all the packet data, ESP parameters, and function pointers to callback functions to execute on success or failure of the hardware request.
2. In addition to any data required by the hardware, keep copies of the *param* and *tdb* buffers. Also, keep a pointer to the IPsec packet, *pkt*. These are needed for iphwcrypto_ipsec_esp_re_input( ) or iphwcrypto_ipsec_esp_re_output( ). The original buffers may not be valid after the ESP callback function returns.
3. Send the request to the hardware.
4. Return one of the following status codes:
   - 2 - asynchronous request submitted
   - -1 - error

When the hardware operation completes successfully it calls the second callback function. This function must perform the following operations:

1. Replace the encrypted payload data with the decrypted data. Value *tdb* **->hdrlen** contains the offset in the buffer pointed to by *tdb* **->data** for the decrypted data. That is, *tdb* **->data** + *tdb* **->hdrlen** points to the location for the data.
2. Retrieve the calculated message digest and the saved values for *tdb*, *param*, and *pkt*. Call iphwcrypto_ipsec_esp_re_input( ) to place the decrypted packet onto the network stack.
3. Clean up any local storage. Do not clean up the packet data, because it is freed after being processed by the network stack.

An output operation is similar, except that it places encrypted data onto the network stack using iphwcrypto_ipsec_esp_re_output( ).

If the hardware operation fails, the callback functions must clean up the saved packet data using iphwcrypto_ipsec_esp_pkt_free( ).

**Parent topic:** Hardware Cryptography Devices on page 18

# 7. FREESCALE SEC ENGINE

The cryptography libraries include support for hardware offload of cryptography operations to Freescale SEC processors, including SEC 3.0 and SEC 3.3.

**Freescale SEC Engine API on page 30**The functions in the Freescale engine API are callback functions of the correct type for use with the hardware cryptography offload API of the cryptography libraries.
**Example Asynchronous AES CCM Encryption Using Scattered Data on page 30**The example performs asynchronous advanced encryption standard (AES) counter with cipher block chaining (CBC) message authentication code (MAC) - (AES CCM) encryption according to IEEE 802.16 using scattered data as input and a single buffer as output.
**freescale Shell Command on page 32**The Freescale engine includes a shell command to test the operation of the Freescale hardware.

## 1. Freescale SEC Engine API

The functions in the Freescale engine API are callback functions of the correct type for use with the hardware cryptography offload API of the cryptography libraries.

The ipfreescale_create( ) function adds and registers all the callback functions for the supported algorithms. This function is run at boot time. The function is located in the following file:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/ipfreescale/src/ ipfreescale.c**

For most applications there is no need to call the Freescale engine API directly. Once the callback functions have been registered with the cryptography libraries, they become the default methods for performing cryptographic operations.

The Freescale engine API can also be used as a reference implementation showing how you should include callback functions into the hardware offload functions of the cryptography libraries.

For applications using the Freescale engine API directly, note the following:

- There is limited support for asynchronous operations with user-supplied callback functions.
- There is limited support for using scattered data for some types of operations.

If you use the Freescale API directly, the **ipfreescale.h** header file must be included by your application. This file is located in:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/ipfreescale/h**

**Parent topic:** Freescale SEC Engine on page 30

## 2. Example Asynchronous AES CCM Encryption Using Scattered Data

The example performs asynchronous advanced encryption standard (AES) counter with cipher block chaining (CBC) message authentication code (MAC) - (AES CCM) encryption according to IEEE 802.16 using scattered data as input and a single buffer as output.

Note: The following example is a conceptual implementation of scattered data.

For further examples, refer to **ipfreescale_cmd_freescale.c** which implements some test commands for supported operations. This file is found in:

*installDir* **/vxworks-7/pkgs_v2/security/crypto_misc-***x.x.x.x* **/ipfreescale/src**

```
/* Call back function for successful jobs */
void async_job_done_cbf(void *handle)
{
    printf("Job with handle=0x%x completed successfully\n", handle);
    /* In this example we use the message location itself as a handle */
    do_something_good_with_message(handle);
}

/* Call back function for jobs with errors */
void async_job_error_cbf(void *handle, int status)
{
    printf("Job with handle=0x%x had errors. status=%d\n", handle, status);
    my_error_handler(handle);
}

void init()
{
    ipfreescale_async_done_cbf_register(&async_job_done_cbf);
    ipfreescale_async_error_cbf_register(&async_job_error_cbf);
}

#define AESCCM_MIC_SIZE 8 /* Size according to IEEE 802.16 */
#define BUFSIZE 2048
unsigned char storeEncyptedMessageHere[BUFSIZE];
unsigned char storeMICHere[AESCCM_MIC_SIZE];

void do_aesccm_encrypt()
{
    char *cleartext1 = "encrypt ";
    char *cleartext2 = "this message";
    unsigned int length = 8 + 12;
    char *key = "0123456789abcdef";
    Ip_u32 keyLen = 16;
    /* Cleartext data as scatter gather table */
    Ipfreescale_sgt_entry sgt[] = {
          {cleartext1, 0, 8, FALSE},
          {cleartext2, 0, 12, TRUE}};

    unsigned char *outData = storeEncyptedMessageHere;
    unsigned char *MICOut = storeMICHere;
    /* In this example we use the message location itself as a handle */
    void *myHandle = (void *)storeEncyptedMessageHere;

    char *inCtx =
        /* AES CCM B0 (initial block) */
        "\x19"                /* AES-CCM flag according to IEEE
                                 * 802.16 */
        "\x40\x40\x1A\x06\xC4" /* First five bytes of gmac */
        "\x00\x00\x00\x00"     /* Reserved */
        "\xBC\xF6\x57\x21"     /* PN (reversed byte order) */
```

```
        "\x00\x10"             /* Cleartext Payload length */
        "\x00\x00\x00\x00\x00\x00\x00\x00"  /* Reserved */
        "\x00\x00\x00\x00\x00\x00\x00\x00"  /* Reserved */
        /* AES CCM Ctr 0 (initial counter block) */
        "\x01"                 /* AES-CCM flag according to IEEE
                                       * 802.16 */
        "\x40\x40\x1A\x06\xC4" /* First five bytes of gmac */
        "\x00\x00\x00\x00"     /* Reserved */
        "\xBC\xF6\x57\x21"     /* PN (reversed byte order)*/
        "\x00\x00"             /* Initial counter */
        "\x00\x00\x00\x00\x00\x00\x00\x00";  /* Reserved */

    int ret;

    init();

    ret = ipfreescale_aesccm
        (
        outData,
        (unsigned char *)&sgt,
        length,
        (unsigned char *)MICOut,
        NULL,
        AESCCM_MIC_SIZE,
        NULL,
        0,
        (void *)key,
        keyLen,
        (unsigned char *)inCtx,
        1,
        OK,
        (void *)myHandle,
        IPFREESCALE_OPT_INPUTDATA_IS_SGT
        );
    if(ret == 2)
        printf("Job sent successfully to SEC engine\n");
    else
        printf("Error in asynchronous call to aesccm encrypt\n");

    /* Continue doing something else while SEC engine perform its business.*/
    ...
    ...
}
```

**Parent topic:** Freescale SEC Engine on page 30

# 3. freescale Shell Command

The Freescale engine includes a shell command to test the operation of the Freescale hardware.

## freescale

Name

**freescale** - Test Freescale hardware operation

Synopsis

```
freescale help
freescale test unit [rounds]
freescale speed unit [rounds]
freescale comp unit [rounds]
freescale stat
freescale clr
freescale sleep secs
```

Description

- **help**
  Print a summary of command options
- **test**
  Test the operation of a named unit or all units. The test is run *rounds* times. If *rounds* is not entered, it defaults to 1.
- **speed**
  Test the speed of a named unit or all units. The test is run *rounds* times. If *rounds* is not entered, it defaults to 1000.
- **comp**
  Compare the speed of a named unit, or all units, to the software implementation. This performs the same test as the **speed** option, but it also runs the software implementation.
- **stat**
  Print statistics.
- **clr**
  Clear statistics.
- **sleep**
  Sleep for *secs* seconds, then return.

**Parent topic:** Freescale SEC Engine on page 30

# 8. SECURE KEY DATABASE - DEPRECATED

The configuration method described herein has been deprecated. However, you are still able to use the methods described.

**Secure Key Database on page 34**
**VSB Options for Secure Key Database on page 35**You must include the **SECURITY_MISC** layer in your VSB or your secure key database functionality is diminished. Without this option, you are only able to store buffers, and not public keys, in the secure key database.
**VIP Components and Parameters for Secure Key Database on page 35**The VIP components allow you to add a key database and shell commands to your project.
**Implementing and Migrating to Secure Key Database Usage on page 35**Implement a secure key database to store encrypted public keys in files.
**Store the Secure Key Database in a ROM File System on page 36**If you do not have a persistent file system, you can store the key database in a ROM file system.
**keydb Shell Command on page 37**The secure key database provides the keydb shell command when the **INCLUDE_IPCOM_KEY_DB_CMD** component is included in your VxWorks kernel image (VIP).

## 1. Secure Key Database

The Secure Key Database is now deprecated but is still currently supported for backwards compatibility.

The Secure Key Database stores public key pairs and passwords/pre-shared keys. It stores the keys encrypted and hides the key-encrypting key. The key-encrypting key is divided into several pieces and distributed in the VxWorks image. At runtime, the key-encrypting key is assembled in order to decrypt the private key. The Secure Key Database utilizes the Cryptography Libraries to encrypt the keys. The encryption algorithm used to encrypt the keys is AES.

In many security protocols (for example, SSH, SSL, and IKE) public key pairs are used. A public key pair consists of a public and a private key. The private key must be stored in a secure way. Furthermore, in some security protocols a so-called pre-shared key can be used (for example, with IKE and WPA). Pre-shared keys are also sensitive information. If the private or pre-shared key is revealed, the security of a system is compromised.

In many traditional server systems, the keys are stored in an encrypted file. When the server is started, an operator is prompted for the key-encrypting key to decrypt the key. This approach is not possible in embedded systems, since there is no operator available.

Without the use of the Secure Key Database, keys in VxWorks are either stored in clear text or encrypted with the key-encrypting key stored in clear text in a file. This means that private keys have no protection in the event that an attacker is able to access the system (for example, a dishonest operator that is able to log on to the system for remote management, an end-user reverse engineering a set-top box, and so on).

When using the Secure Key Database, each key is identified by a key identifier. The key identifier is a text string.

Each key is stored encrypted in a file, and each key is stored in a separate file. The file name is the key identifier. Key pairs and pre-shared keys are stored in separate directories. Key pairs and pre-shared keys have separate namespaces.

The key-encrypting key is set at build time. Each customer must set a unique key-encrypting key.

For configuration information, see VIP Components and Parameters for Secure Key Database on page 35.

> Note: The VxWorks C interpreter shell should not be included in deployed systems when the Secure Key Database is used. Using the VxWorks C interpreter shell, it is trivial to output the keys in cleartext.

**Parent topic:** Secure Key Database - Deprecated on page 34

# 2. VSB Options for Secure Key Database

You must include the **SECURITY_MISC** layer in your VSB or your secure key database functionality is diminished. Without this option, you are only able to store buffers, and not public keys, in the secure key database.

Note: When using the secure key database, do not include the C interpreter in the shell in a deployed system. The VxWorks C interpreter shell can be used to output the keys in clear text, thereby undermining the security of the system.

**Parent topic:** Secure Key Database - Deprecated on page 34

# 3. VIP Components and Parameters for Secure Key Database

The VIP components allow you to add a key database and shell commands to your project.

| Components and Parameters | Description |
|---|---|
| **INCLUDE_IPCOM_USE_KEY_DB** | This is the IPCOM key database, which stores keys in an encrypted format. |
| **INCLUDE_IPCOM_KEY_DB_CMD** | This is a shell command to manipulate the IPCOM key database. For more information about the keydb shell command, see keydb Shell Command on page 37. |

**Parent topic:** Secure Key Database - Deprecated on page 34

# 4. Implementing and Migrating to Secure Key Database Usage

Implement a secure key database to store encrypted public keys in files.

## About This Task

The following procedure assumes that you are currently using public keys but are not using the secure key database.

## Procedure

1. When using the secure key database, the file system must be persistent. Otherwise, the keys are lost at reboot. To migrate to, or implement, the secure key database, set the configuration parameter, **IPCOM_KEY_DB_FILE_ROOT** to **"/flash/keys"**.

   By default, this parameter is set to match the default value of **IPCOM_FILE_ROOT**, which is **"/ram"**. For more information, see VIP Components and Parameters for Secure Key Database on page 35.

2. Install new keys under the IDs of **default_rsa** and **default_dsa**, accordingly.



WNDRVR

By doing this, you do not need to reconfigure users of the secure key database, such as SSH. By default, SSH also has its key ID parameters set to **default_rsa** and **default_dsa**, respectively. This is also true for other users of the secure key database.

3. Include the components **INCLUDE_IPCOM_USE_KEY_DB** and set the associated configuration parameters **IPCOM_KEY_DB_KEY_ENCRYPTING_KEY** and **IPCOM_KEY_DB_FILE_ROOT** in your VxWorks image project.
4. Rebuild your image.
5. Populate the key database by importing the RSA and DSA keys.

   You can import the RSA and DSA keys in one of the following ways:

   - Use the keydb shell command. To use the keydb command, you must include **INCLUDE_IPCOM_KEY_DB_CMD** in your VIP. See keydb Shell Command on page 37.
   - Write code that imports the keys by calling ipcom_key_db_pkey_import( ) and/or ipcom_key_db_buf_import( ), depending on the type of key to import. For more information on these functions, refer to the ipcom_key_db( ) API in the VxWorks 7 Kernel Networking API Reference.

   Give these keys the key IDs mentioned previously. Internally, the secure key database stores the keys encrypted in files.

6. Reboot your system.

   Because the key database root directory is in a persistent file system, the key files remain after a reboot.

### Results

SSH starts and uses the keys stored in the secure key database.

**Parent topic:** Secure Key Database - Deprecated on page 34

## 5. Store the Secure Key Database in a ROM File System

If you do not have a persistent file system, you can store the key database in a ROM file system.

### About This Task

The disadvantage to this approach is that the keys in the key database are hardcoded and, thus, cannot be changed without upgrading the system.

To store the key database in a ROM file system, proceed as follows:

### Procedure

1. Configure and build your VxWorks image project (VIP).

   Add the **INCLUDE_IPCOM_USE_RAM_DISK** and **INCLUDE_IPCOM_KEY_DB_CMD** components to your VIP and set the IPCOM_KEY_DB_FILE_ROOT parameter to **"/ram"**.

2. Download the image to your target and boot your system.
3. Download and import the keys.



WNDRVR

   1. Download the keys you want to import to your system and import the keys into the secure key database using the keydb shell command. See keydb Shell Command on page 37.
   2. Download the **"/ram/pkey"** and/or **"/ram/buf"** to your host.
4. Modify your project configuration.
   1. Remove the **INCLUDE_IPCOM_USE_RAM_DISK** and **INCLUDE_IPCOM_KEY_DB_CMD** components from your VIP.
   2. Add the **INCLUDE_ROMFS** component to your VIP.
   3. Change **IPCOM_KEY_DB_FILE_ROOT** to **/romfs**.
   4. Copy the contents of **/ram/pkey** and/or **/ram/buf** to *vipDir* **/romfs/pkey** and *vipDir* **/romfs/buf**, respectively.
5. Rebuild the VIP project.
6. Reboot your system.

Once you reboot your target, the secure key database is stored in the ROM filesystem.

**Parent topic:** Secure Key Database - Deprecated on page 34

# 6. keydb Shell Command

The secure key database provides the keydb shell command when the **INCLUDE_IPCOM_KEY_DB_CMD** component is included in your VxWorks kernel image (VIP).

## keydb

The legacy secure key database and the **keydb** shell command are supported for backwards compatibility reasons, but are deprecated.

Name

**keydb** - Provides shell commands for working with the secure key database.

Synopsis

```
keydb [options]
    list <key|buf>
    import key key_id <key_file>
    import buf key_id buf
    delete <key|buf> <id>
    print <key_id>
    find <cert_file>
    verify <buf_id>
    help|-h
```

Description

- **list** *<key|buf>*

  Returns the public key with the specified key ID *<key_id>*.
- **import key key_id** *<key_file>*

  Imports a public key pair into the key database.
- **import buf key_id** *buf*

  Imports a buffer into the key database and gives it the ID, *key_id*. The buffer is typically a password or a pre-shared key.
- **delete** *<key|buf> <id>*



WNDRVR

Deletes the buffer or key with the ID *id* from the key database.

- **print** *<key_id>*
  Prints the specified *key_id*. Sensitive information is not printed.
- **find** *<cert_file>*
  Searches the key database for a public key pair matching the public key in the specified X.509 certificate.
- **verify** *<buf_id>*
  Verifies the contents of a prompted key with the value of the key in the buffer with ID *buf_id*.
- **help|-h**
  Lists the available keydb commands.

**Parent topic:** Secure Key Database - Deprecated on page 34